CSC209 Assignment 3
Multi-Process Monte Carlo π Estimator (Pipes)

<team name on MarkUs>
<name, student number> (repeat per member)

April 2, 2026

1 Project Overview

1.1 Category and purpose

This project is a **Category 1** systems program: a parent *controller* process coordinates multiple *worker* child processes using **pipes** for all substantive inter-process communication. The application domain is a Monte Carlo estimator for π: workers draw uniform random points in $[0, 1)^2$ and count hits under $x^2 + y^2 \leq 1$; the controller aggregates hits and trials and reports an estimate, a normal-approximation 95% confidence interval, and error versus `M_PI`.

The emphasis is on **process orchestration**, **typed binary protocols**, **concurrency**, and **robustness**—not on algorithmic novelty.

1.2 User interaction

The controller reads commands from standard input:

- `simulate <N>` — run N trials across alive workers (work is split and results may arrive in **multiple partial messages** per worker).
- `setbatch <N>` — set the maximum trials per partial `RESULT_SIMULATE` message (increases message *frequency* for the same N when batches are small).
- `stats` — query each worker's cumulative simulate-task counter (separate control-plane traffic).
- `workers` — print alive worker count.
- `help` / `quit` (or `exit`).

1.3 Why this satisfies "communication complexity"

The wire format is not a single opaque payload per job. The design exposes:

- **Multiple message types** (simulate, status, shutdown, error responses).
- **Versioned structs** (`PROTOCOL_VERSION`) so both ends agree on layout and semantics.
- **High-frequency exchanges** when batch sizes are small: one simulate command triggers many `RESULT_SIMULATE` messages.
- **Explicit batch metadata** (`batch_index`, `batch_total`) so the controller can validate ordering and completeness.

1.4 Mathematical note

For uniform samples in the unit square, the quarter-disk area is $\pi/4$, so $\hat{\pi} \approx 4 \cdot (\text{hits/trials})$.

2 Build and run instructions

2.1 Environment

- Language: C (GNU/Linux, e.g. `teach.cs`).
- Compiler: `gcc`; build: `make`.
- Flags: `-Wall -Wextra -g`; link math: `-lm` for `controller`.
- No third-party libraries beyond the standard toolchain.

2.2 Commands

```
make clean
make
./controller [num_workers [seed]]
```

Defaults: four workers, base seed 42. Constraints: `num_workers` $\in [3, 64]$.

2.3 Typical session

```
./controller 4 42
# then at the prompt:
help
stats
setbatch 20000
simulate 500000
workers
quit
```

2.4 Optional demo script

`demo_error.sh` rebuilds the project, runs the controller with piped input, kills one worker, and shows that later `simulate` commands still succeed with fewer alive workers.

2.5 Note on timing metrics

The controller prints wall-clock duration and throughput using `clock_gettime(CLOCK_MONOTONIC, ...)`. This is an **optional observability aid**; core grading mechanisms remain `fork`, `exec`, `pipe`, `read`/`write`, `waitpid`, and signals.

3 Architecture

Figure 1 shows processes and pipes. Each worker has two dedicated unidirectional pipes: **task** (controller→worker) and **result** (worker→controller).

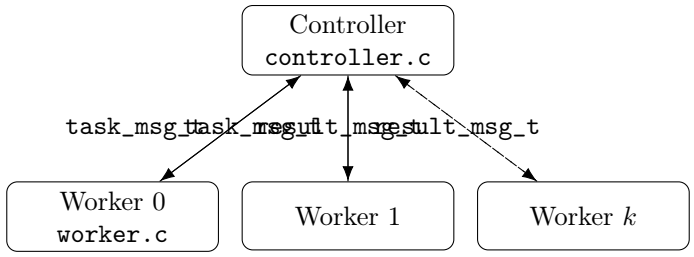

Figure 1: Worker pool: one controller, N workers, two pipes per worker.

3.1 Key source files

File	Role
`montecarlo.h`	Shared protocol constants and `task_msg_t` / `result_msg_t`.
`controller.c`	`spawn_workers`, `run_simulation`, `query_worker_stats`, `shutdown_workers`, REPL.
`worker.c`	Read loop, Monte Carlo trials, batched responses, status/error replies.
`Makefile`	`make` builds `controller` and `worker` with `-Wall -Wextra`.

4 Communication protocol

All messages are **fixed-width binary structs** sent with a single `write` of `sizeof(...)` bytes per message. Partial reads are treated as failure. Helpers `read_full` and `write_full` retry on `EINTR`.

4.1 Request: `task_msg_t` (controller → worker)

```
typedef struct {
    uint32_t version;
    uint32_t task_id;
    uint32_t msg_type;
    uint32_t num_trials;
    uint32_t batch_trials;
} task_msg_t;
```

`msg_type` is one of `TASK_SIMULATE`, `TASK_STATUS_REQ`, or `TASK_SHUTDOWN`. Fields `num_trials` and `batch_trials` apply to `TASK_SIMULATE` (batch cap for partial results).

4.2 Response: `result_msg_t` (worker → controller)

```
typedef struct {
    uint32_t version;
    uint32_t task_id;
    uint32_t msg_type;
    uint32_t num_trials;
    uint32_t num_hits;
    uint32_t batch_index;
    uint32_t batch_total;
    uint32_t tasks_completed;
    uint32_t error_code;
```

```
} result_msg_t;
```

`msg_type` is `RESULT_SIMULATE`, `RESULT_STATUS`, or `RESULT_ERROR`. For simulate batches, `batch_index` runs 1 . . . `batch_total`; the controller checks monotonicity and that the sum of `num_trials` matches the chunk assigned to that worker.

4.3 Protocol matrix (summary)

Exchange	When	Notes
`TASK_SIMULATE`	User `simulate`	Carries chunk size and `batch_trials`.
`RESULT_SIMULATE`	Worker computes	One or many per task; includes batch indices.
`TASK_STATUS_REQ`	User `stats`	Control traffic distinct from simulate.
`RESULT_STATUS`	Worker replies	Exposes `tasks_completed`.
`TASK_SHUTDOWN`	User `quit`	Clean termination path.
`RESULT_ERROR`	Invalid request / version	Worker-side error reporting.

4.4 Invariants

- `version` must equal `PROTOCOL_VERSION` on both sides.
- `task_id` on responses must match the outstanding request the controller is aggregating.
- For `RESULT_SIMULATE`: `num_hits` $\leq$ `num_trials`; batch metadata must match the expected batch count derived from chunk and `batch_trials`.

A reader could re-implement the wire protocol from this section plus `montecarlo.h` without reading the control flow.

5 Concurrency model

5.1 Worker pool

The controller calls `spawn_workers` before the REPL: it creates two pipes per worker, `fork`s, and the child `execl`s `./worker` with the read end of the task pipe, the write end of the result pipe, and a per-worker seed. This yields **at least three concurrent workers** whenever `num_workers` ≥ 3 (enforced in `main`).

5.2 Synchronisation

Workers block on `read` from the task pipe until the controller sends a message—classic producer/-consumer synchronisation via the OS pipe buffer.

5.3 Simulate path

For `simulate N`, the controller counts alive workers, partitions N with a remainder distribution, and writes one `TASK_SIMULATE` per worker that receives non-zero work. It then reads `RESULT_SIMULATE` messages until each worker's assigned trials are fully accounted for. Partial reads or protocol violations mark that worker dead and close its fds so later commands skip broken peers.

5.4 Relation to assignment wording

For very small N, some workers may be idle for that command; the **pool remains concurrently resident**. For reasonable N, all workers participate, matching the forum clarification that concurrency should hold for non-trivial workloads.

5.5 Signals

`SIGPIPE` is ignored in the controller so broken pipes surface as `EPIPE` on `write`. A `SIGCHLD` handler reaps terminated children with `waitpid(..., WNOHANG)` and clears `alive` flags; heavy cleanup uses the same fd helpers as the main line.

6 Error handling and robustness

6.1 Case 1: partial startup failure

If `pipe` or `fork` fails mid-`spawn_workers`, the controller runs a **rollback**: closes already-open pipe fds, terminates and reaps any children already created, resets bookkeeping, and returns failure so `main` exits cleanly. This avoids orphan processes and fd leaks on failed launch.

6.2 Case 2: broken pipe while dispatching

If `write_full` fails when sending a task (including `EPIPE`), the controller calls `mark_worker_dead`: clear `alive` and close that worker's controller-side fds. Other workers continue to serve later `simulate` and `stats` commands.

6.3 Case 3: bad or incomplete results

Short reads, wrong `task_id`, wrong `msg_type`, version mismatch, invalid batch metadata, or trial counts exceeding the assigned chunk all trigger the same policy: mark the worker dead, discard bad data, continue aggregation from healthy workers. This prevents one faulty peer from corrupting global statistics.

6.4 Case 4: unexpected child exit

The `SIGCHLD` handler reaps exited workers so they do not linger as zombies during long interactive sessions.

6.5 Resource management

`close_worker_fds` and `mark_worker_dead` centralise fd lifecycle. `shutdown_workers` sends `TASK_SHUTDOWN`, closes fds, and `waitpid`s each known child. The core path avoids heap allocation, reducing leak surface.

7 Team contributions

Replace this section with one short paragraph per member (name and student number), describing concrete files and responsibilities (e.g. protocol design, `controller.c` orchestration, `worker.c` batching, Makefile, report, demo script).

Complexity justification (rubric alignment)

This project keeps the **application layer** simple (Monte Carlo) so effort concentrates on **systems programming**: a versioned, multi-type pipe protocol with optional high message frequency (batching + `setbatch`), explicit shutdown, status queries, startup rollback, signal-aware IO, and graceful degradation when workers die. Together these address the course emphasis on *encoding*, *message diversity*, *exchange frequency*, and *clear sender/receiver roles*.